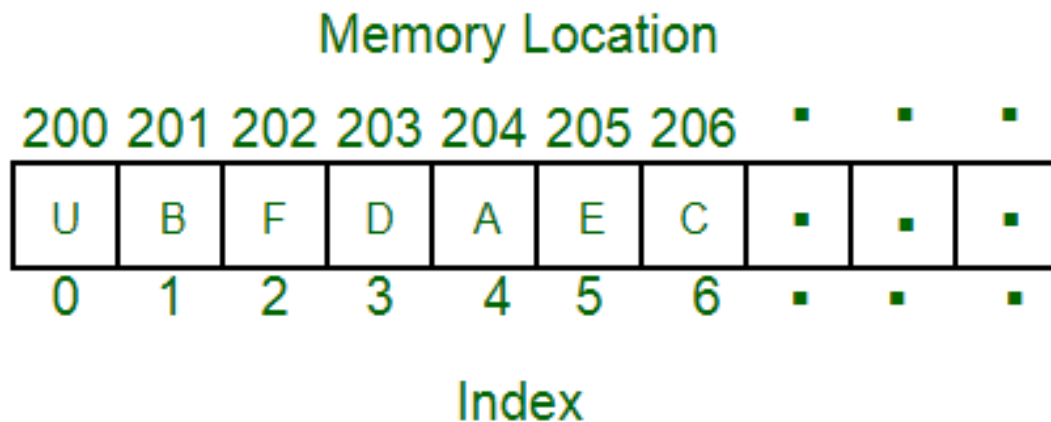


# Data Structure and Algorithms Lecture

Mamati Barasa G

# What is Data Structure?

- Data structure is a representation of data and the operations allowed on that data.
- A data structure is a particular way of organizing data in a computer so that it can be used effectively.
- For example, we can store a list of items having the same data-type using the array data structure.



➤ **Data Structure mainly specifies the following four things:**

- 1)organization of data
- 2)accessing method
- 3)degree of associativity
- 4) processing alternative for information

➤ **Algorithm + Data Structure = Program**

➤ **Data Structure study Covers the following points**

- 1) Amount of memory require to store
- 2) Amount of time require to process
- 3) Representation of data in memory
- 4) Operations performs on data

# Types of Data Structures

- The DS are divided into two types:
  - 1) Primitive/Built in DS
  - 2) Non primitive/User Defined DS
- Non primitive/User Defined DS divided into two type
  - 1) Linear DS
  - 2) Non linear DS

# Operation on Data Structures

- **Create:** This operation results in reserving memory for program elements. This can be done by declaration statement Creation of DS may take place either during compile-time or run-time.
- **Destroy:** This operation destroy memory space allocated for specified data structure .
- **Selection:** This operation deals with accessing a particular data within a data structure.
- **Updation:** It updates or modifies the data in the data structure.

- **Searching:** It finds the presence of desired data item in the list of data items, it may also find locations of all elements that satisfy certain conditions.
- **Sorting:** This is a process of arranging all data items in a DS in particular order, for example either ascending order or in descending order.
- **Splitting:** It is a process of partitioning single list to multiple list.
- **Merging:** It is a process of combining data items of two different sorted list into single sorted list.
- **Traversing:** It is a process of visiting each and every node of a list in systematic manner.

# Primitive Data Structure

- Primitive Data Structure are basic structure and directly operated upon by machine instructions.
- These data types are available in most programming languages as built in type.

**Integer:** -whole number

**Float:** fraction/decimal numbers

**Character:** Character values

**Pointer:** A variable that hold memory address

# Non-Primitive DS

- These are more sophisticated data structures.
- The non – primitive data structures emphasize structuring of a group of homogeneous or heterogeneous data items.
- A non – primitive data type is further divided into Linear and non – Linear data structure.
- Linear DS: Arrays, lists, stacks, queues
- Non Linear DS: Trees, graphs, hash tables

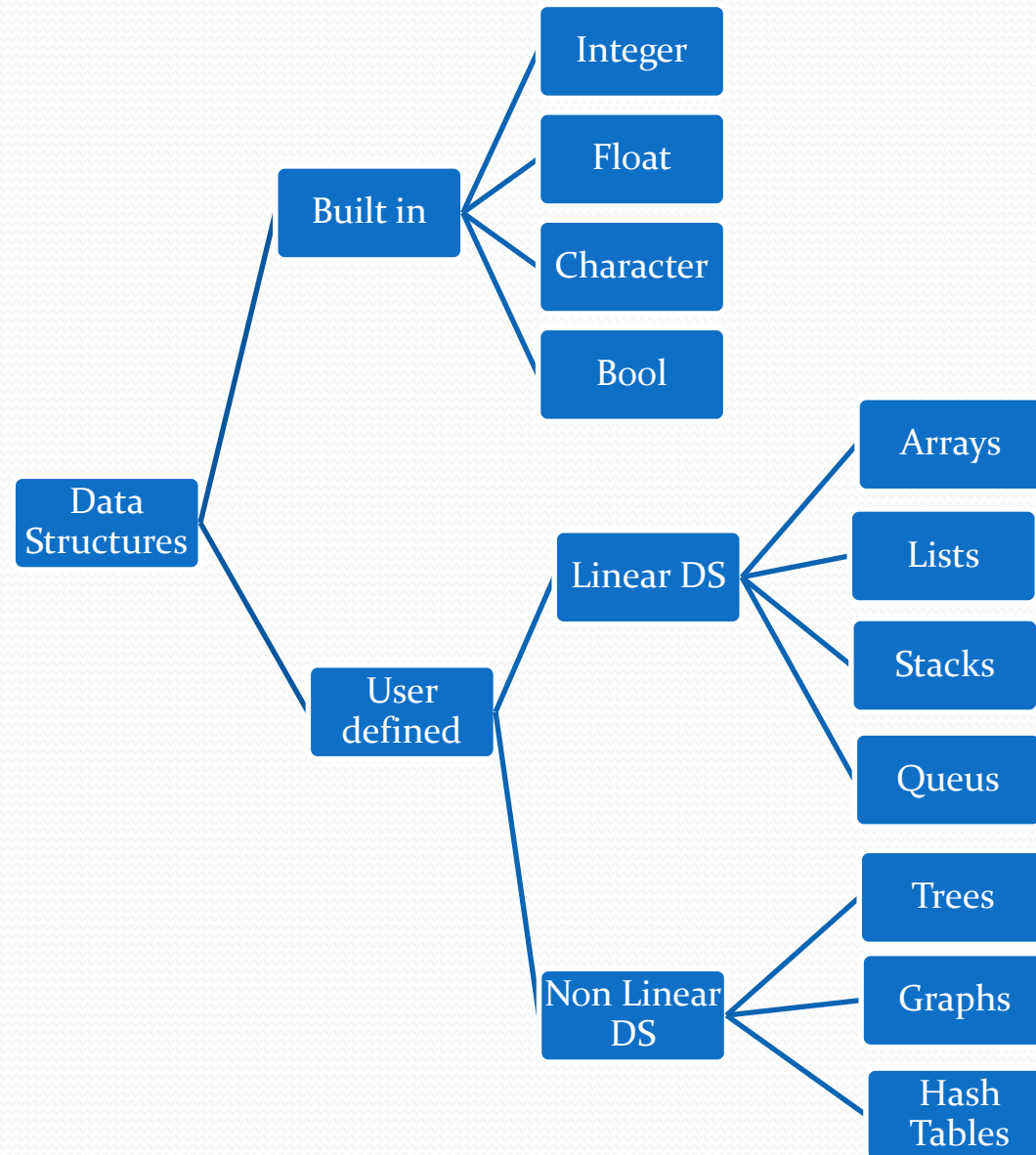
# Linear data structures

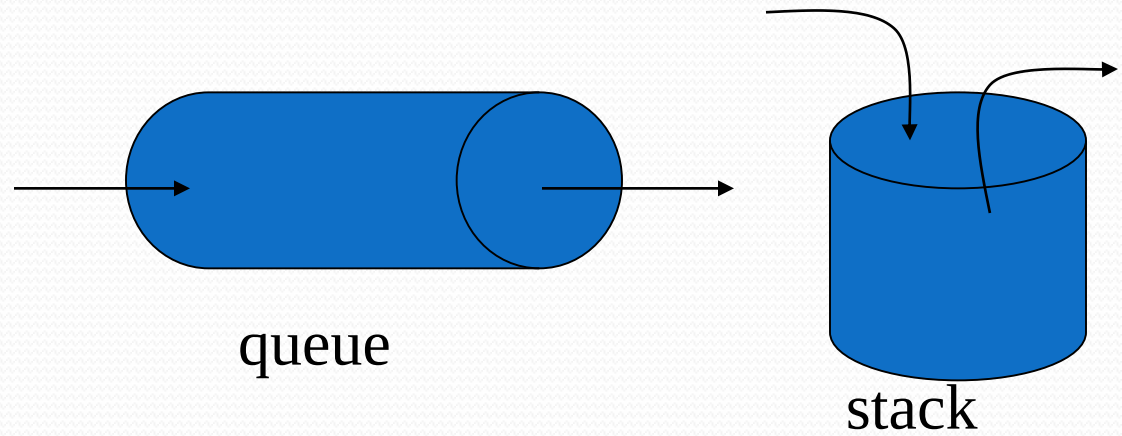
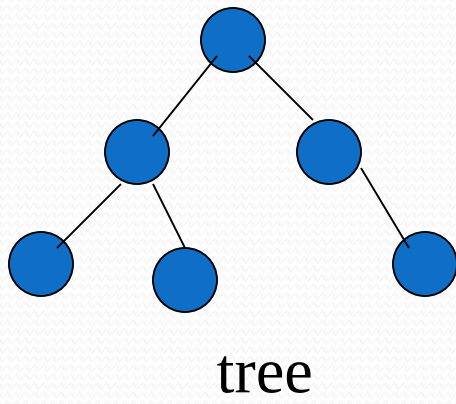
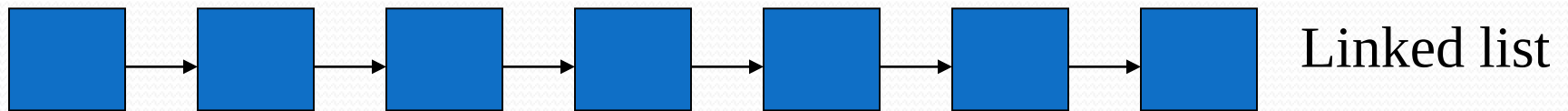
- A linear data structure simply means that it is a storage format of the data in the memory in which the data are arranged in continuous blocks of memory. E.g Arrays-one character after another
- member elements form a sequence in the storage.
- ways to represent a linear DS in memory.
  - **static memory allocation**
  - **dynamic memory allocation**

possible operations:1) Traversing 2) Insertion 3) Deletion 4) searching 5) sorting 6) merging

# Selection of Data Structure

- The choice of particular data model depends on two consideration:
  - It must be rich enough in structure to represent the relationship between data elements
  - The structure should be simple enough that one can effectively process the data when necessary





# Types of Data Structure

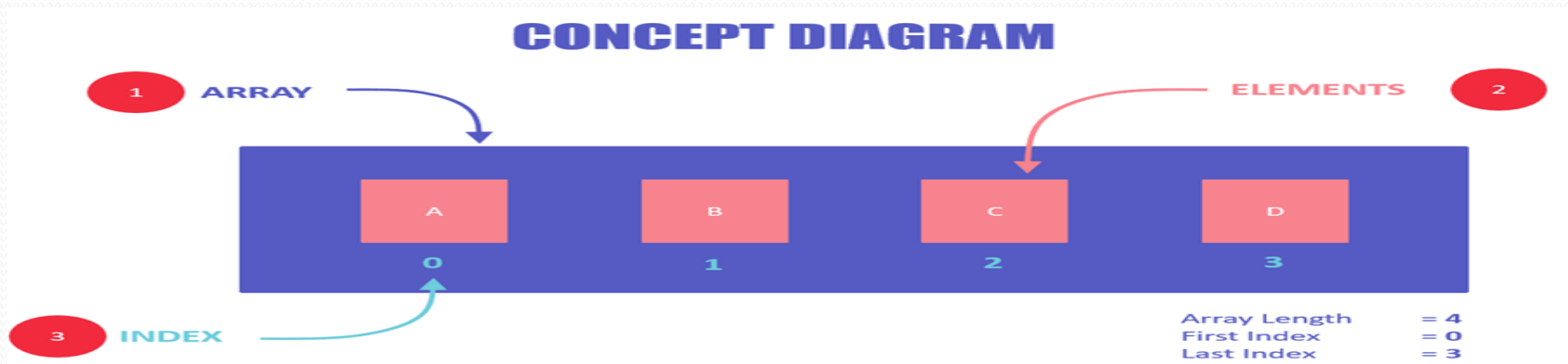
- Linear: In Linear data structure, values are arranged in linear fashion.
  - Array: Fixed-size
  - Linked-list: Variable-size
  - Stack: Add to top and remove from top: LIFO
  - Queue: Add to back and remove from front: FIFO
  - Priority queue: Add anywhere, remove the highest priority
- Non-Linear: The data values in this structure are not arranged in order.
  - Hash tables: Unordered lists which use a 'hash function' to insert and search
  - Tree: Data is organized in branches.
  - Graph: A more general branching structure, with less strict connection conditions than for a tree

# Arrays

- Array is a container which can hold a fix number of items and these items should be of the same type.
- Key terms

**Element** – Each item stored in an array is called an element.

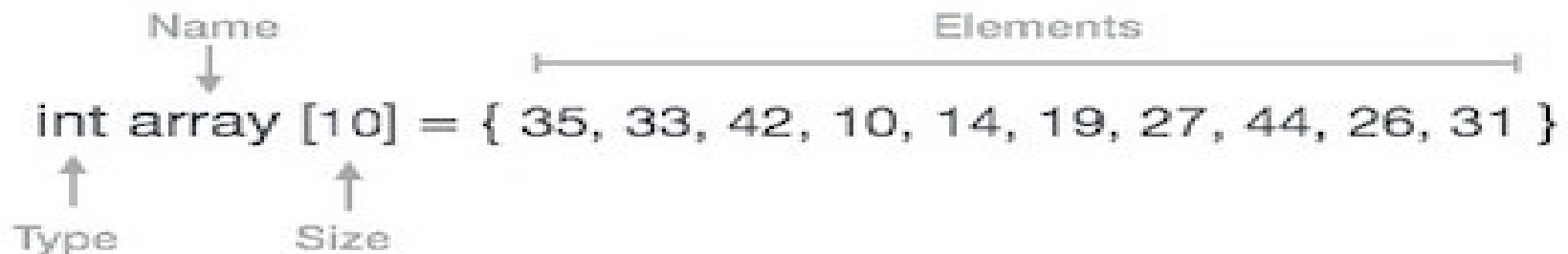
**Index** – Each location of an element in an array has a numerical index, which is used to identify the element.



# Declaring Arrays

- three parts will always remain common in all the initializations, i.e.,
  - data type of elements
  - array name,
  - Size of array
  - Elements
- You can access any array item by using its index from 0 to n elements

```
int age[6];
```



The diagram illustrates the components of the array declaration `int array [10] = { 35, 33, 42, 10, 14, 19, 27, 44, 26, 31 };`. Labels with arrows point to specific parts: 'Name' points to 'array', 'Type' points to 'int', 'Size' points to '[10]', and 'Elements' points to the list of values in curly braces.

```
int array [10] = { 35, 33, 42, 10, 14, 19, 27, 44, 26, 31 };
```

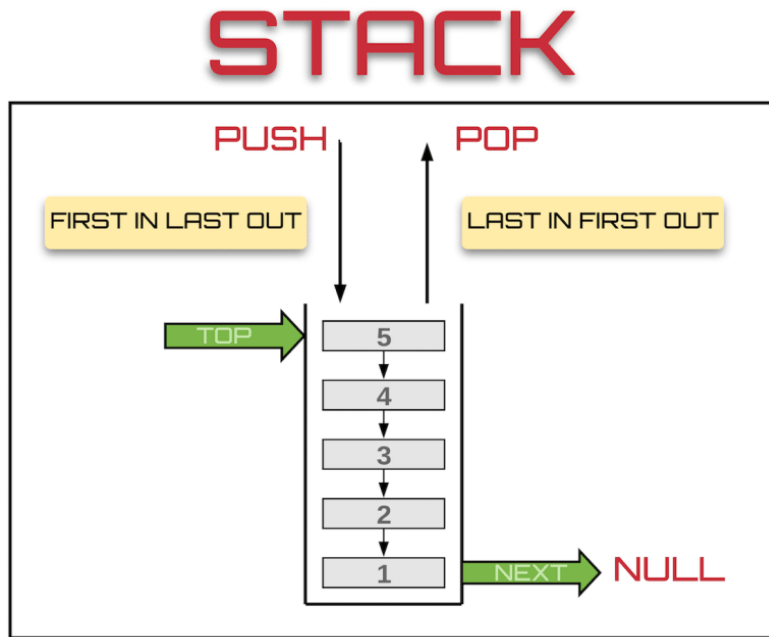
# Basic Operations on Arrays

- Following are the basic operations supported by an array.
- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.

# Applications of Arrays

- Arrays are used to implement data structures like a stack, queue, etc.
- Arrays are used for matrices and other mathematical implementations.
- Arrays are used in lookup tables in computers.
- Arrays can be used for CPU scheduling

# Stacks



Stack is a data structure in which insertion and deletion operations are performed at one end only. The insertion operation is referred to as 'PUSH' and deletion is referred as 'POP' operation

- Stack is also called as Last In First Out (LIFO) data structure.

# Basic Operations on Stacks

- **push()** – Pushing (storing) an element on the stack.
- **pop()** – Removing (accessing) an element from the stack.
- **peek()** – get the top data element of the stack, without removing it.
- **isFull()** – check if stack is full.
- **isEmpty()** – check if stack is empty.

# Push Operation

- The process of putting a new data element onto stack is known as a Push Operation. Push operation involves a series of steps –
- **Step 1** – Checks if the stack is full.
- **Step 2** – If the stack is full, produces an error and exit.
- **Step 3** – If the stack is not full, increments **top** to point next empty space.
- **Step 4** – Adds data element to the stack location, where top is pointing.
- **Step 5** – Returns success.

# Pop Operation

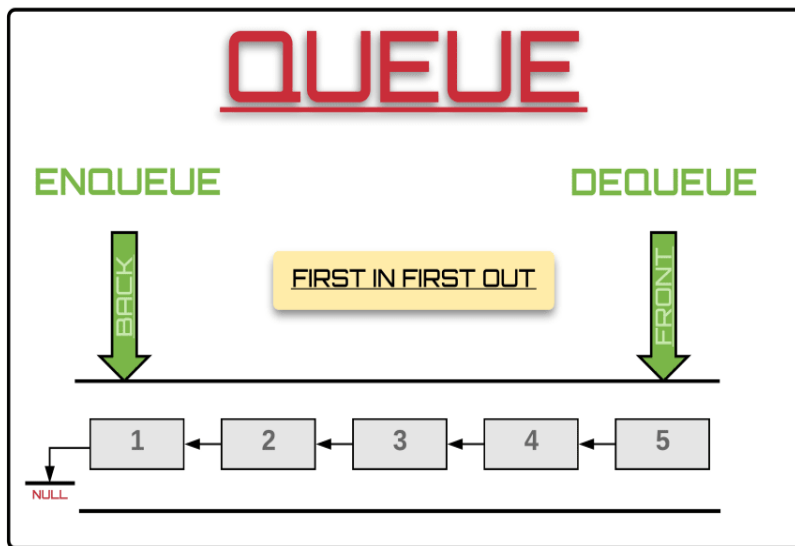
- A Pop operation may involve the following steps –
- **Step 1** – Checks if the stack is empty.
- **Step 2** – If the stack is empty, produces an error and exit.
- **Step 3** – If the stack is not empty, accesses the data element at which **top** is pointing.
- **Step 4** – Decreases the value of top by 1.
- **Step 5** – Returns success.

# Application of stacks

- Stack is used for evaluating expression with operands and operations.
- Matching tags in HTML and XML
- Undo function in any text editor.
- Infix to Postfix conversion.
- Stacks are used for backtracking and parenthesis matching.
- Stacks are used for conversion of one arithmetic notation to another arithmetic notation.
- Stacks are useful for function calls, storing the activation records and deleting them after returning from the function. It is very useful in processing the function calls.
- Stacks help in reversing any set of data or strings.

# Queues

- The data structure which permits the insertion at one end and deletion at another end, known as Queue.
- End at which deletion is occurs is known as FRONT end and another end at which insertion occurs is known as REAR end.
- Queue is also called as First In First Out (FIFO)



# Basic Operations on Queues

- **enqueue()** – add (store) an item to the queue.
- **dequeue()** – remove (access) an item from the queue.
- **peek()** – Gets the element at the front of the queue without removing it.
- **isfull()** – Checks if the queue is full.
- **isempty()** – Checks if the queue is empty

# Enqueue Operation

The following steps should be taken to enqueue (insert) data into a queue

- **Step 1** – Check if the queue is full.
- **Step 2** – If the queue is full, produce overflow error and exit.
- **Step 3** – If the queue is not full, increment **rear** pointer to point the next empty space.
- **Step 4** – Add data element to the queue location, where the rear is pointing.
- **Step 5** – return success

# Deque Operation

The following steps are taken to perform **dequeue** operation –

- **Step 1** – Check if the queue is empty.
- **Step 2** – If the queue is empty, produce underflow error and exit.
- **Step 3** – If the queue is not empty, access the data where **front** is pointing.
- **Step 4** – Increment **front** pointer to point to the next available data element.
- **Step 5** – Return success.

# Application of Queues

- When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.
- When data is transferred asynchronously (data not necessarily received at the same rate as sent) between two processes. Examples include IO Buffers, pipes, etc
- Semaphores
- FCFS ( first come first serve) scheduling, example: FIFO queue
- Spooling in printers
- Buffer for devices like keyboard
- Queues in routers/ switches
- Mail Queues
- **Variations:** ( Deque, Priority Queue, Doubly Ended Priority Queue
- **Some other applications of Queue:**
- Applied as waiting lists for a single shared resource like CPU, Disk, and Printer.
- Applied as buffers on MP3 players and portable CD players.
- Applied on Operating system to handle the interruption.
- Applied to add song at the end or to play from the front.
- Applied on WhatsApp when we send messages to our friends and they don't have an internet connection then these messages are queued on the server of WhatsApp

# Linked Lists

- A linked list is a sequence of data structures, which are connected together via links.
- Linked List is a sequence of links which contains items. Each link contains a connection to another link.

## **important terms**

- **Link** – Each link of a linked list can store a data called an element.
- **Next** – Each link of a linked list contains a link to the next link called Next.
- **LinkedList** – A Linked List contains the connection link to the first link called First.

# Linked List Representation

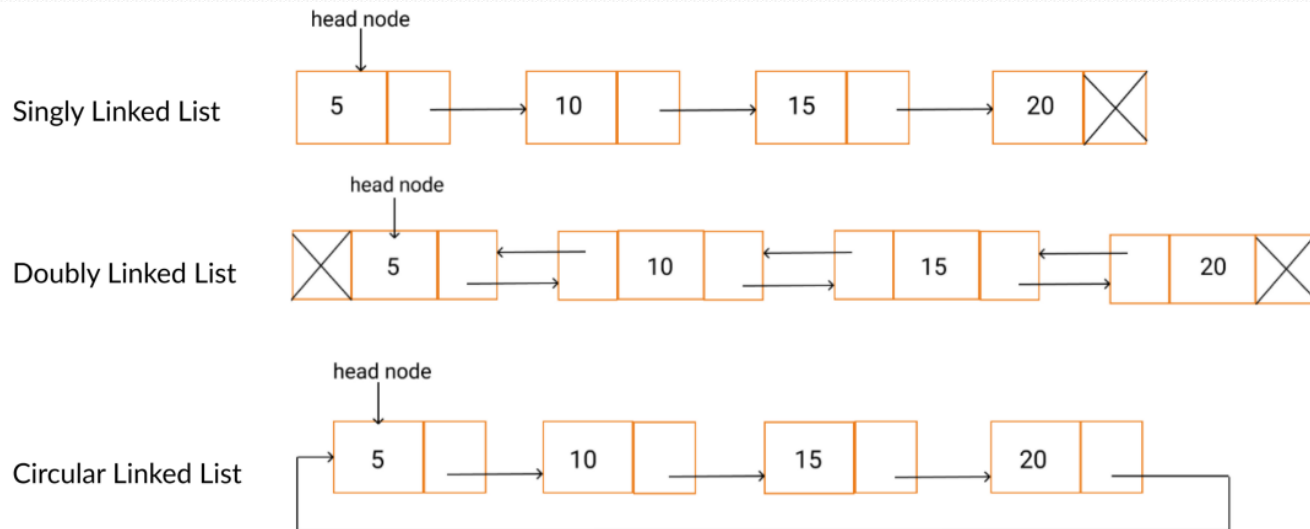
- Linked list can be visualized as a chain of nodes, where every node points to the next node



- Linked List contains a link element called first.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.

# Types of Linked List

- Following are the various types of linked list.
- **Simple Linked List** – Item navigation is forward only.
- **Doubly Linked List** – Items can be navigated forward and backward.
- **Circular Linked List** – Last item contains link of the first element as next and the first element has a link to the last element as previous.



# Basic operations on linked lists

- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Display** – Displays the complete list.
- **Search** – Searches an element using the given key.
- **Delete** – Deletes an element using the given key.

# Example operations

- **Add Operation**

- e.g.  $\text{Add}(1, X, L)$  adds the value  $X$  to list  $L$  at position 1 (the start of the list is position 0), shifting subsequent elements up  $L$

List L1 Before 

|   |   |   |
|---|---|---|
| A | B | C |
|---|---|---|

List L2 After 

|   |   |   |   |
|---|---|---|---|
| A | X | B | C |
|---|---|---|---|

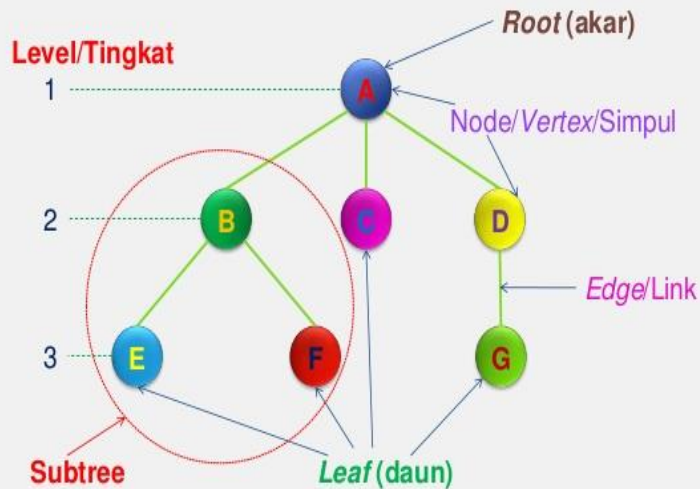
- Set: e.g.  $\text{Set}(2, Z, L)$  updates the values at position 2 to be
- List L3 before: **A X B C**                      List L3 after **A X Z C**
- Remove: e.g.  $\text{Remove}(Z, L)$  removes the node with value  $Z$
- List L4 before: **A X B C**                      List L4 after **A X C**
- $\text{Get}(2, L_4)$  returns the value of the third node=B

# Examples of Lists

- Implementation of stacks and queues
- Implementation of graphs: Adjacency list representation of graphs is the most popular which uses a linked list to store adjacent vertices.
- Dynamic memory allocation: We use a linked list of free blocks.
- Maintaining a directory of names
- Performing arithmetic operations on long integers
- Manipulation of polynomials by storing constants in the node of the linked list
- representing sparse matrices

# Trees

## Components of Tree



- A Tree can be define as finite data items (nodes) in which data items are arranged in branches and sub branches
- Tree represent the hierarchical relationship between various elements
- A tree data structure is a non-linear data structure

# Terms in Tree DS

- **Root:** The root of a tree is the topmost node of the tree that has no parent node. There is only one root node in every tree.
- **Edge:** Edge acts as a link between the parent node and the child node.
- **Leaf:** A node that has no child is known as the leaf node. It is the last node of the tree. There can be multiple leaf nodes in a tree.
- **Subtree:** The subtree of a node is the tree considering that particular node as the root node.
- **Depth:** The depth of the node is the distance from the root node to that particular node.
- **Height:** The height of the node is the distance from that node to the deepest node of that subtree.
- **Height of tree:** The Height of the tree is the maximum height of any node. This is same as the height of root node

# Types of Tree data structure

- **General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum  $n$  number of nodes
- **Binary tree:** Here, binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes
- **Binary search tree:** A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

# Binary Tree Traversals:

- **PreOrder Traversal:** Here, the traversal is: root – left child – right child. It means that the root node is traversed first then its left child and finally the right child.
- **InOrder Traversal:** Here, the traversal is: left child – root – right child. It means that the left child is traversed first then its root node and finally the right child.
- **PostOrder Traversal:** Here, the traversal is: left child – right child – root. It means that the left child is traversed first then the right child and finally its root node.

- Let us traverse the following tree with all the three traversal methods:
- Tree
- 1 //Root Node
- / \
- 2   3
- / \ / \
- 4 5 6 7 //Leaf Nodes
- **PreOrder Traversal of the above tree: 1-2-4-5-3-6-7**
- **InOrder Traversal of the above tree: 4-2-5-1-6-3-7**
- **PostOrder Traversal of the above tree: 4-5-2-6-7-3-1**

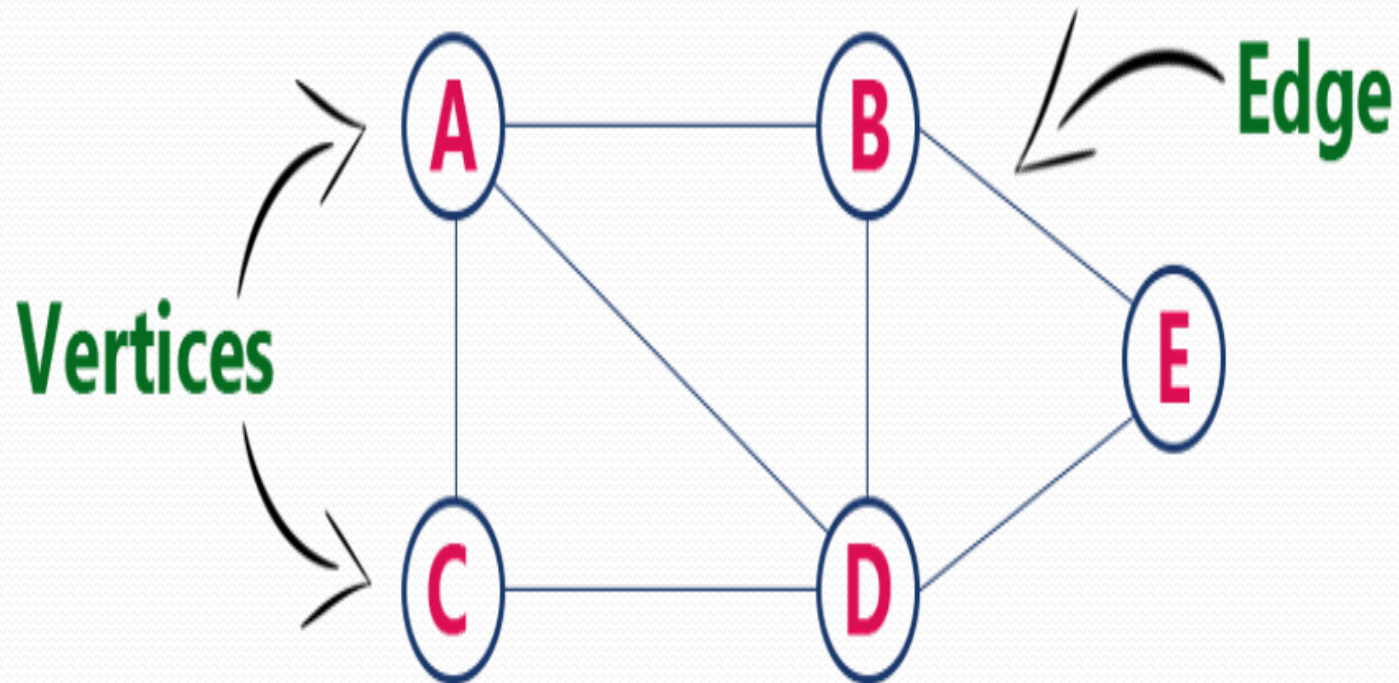
# Application of Tree DS

- Manipulate hierarchical data.
- Make information easy to search (see tree traversal).
- Manipulate sorted lists of data.
- As a workflow for compositing digital images for visual effects.
- Router algorithms
- Form of multi-stage decision-making (see business chess).

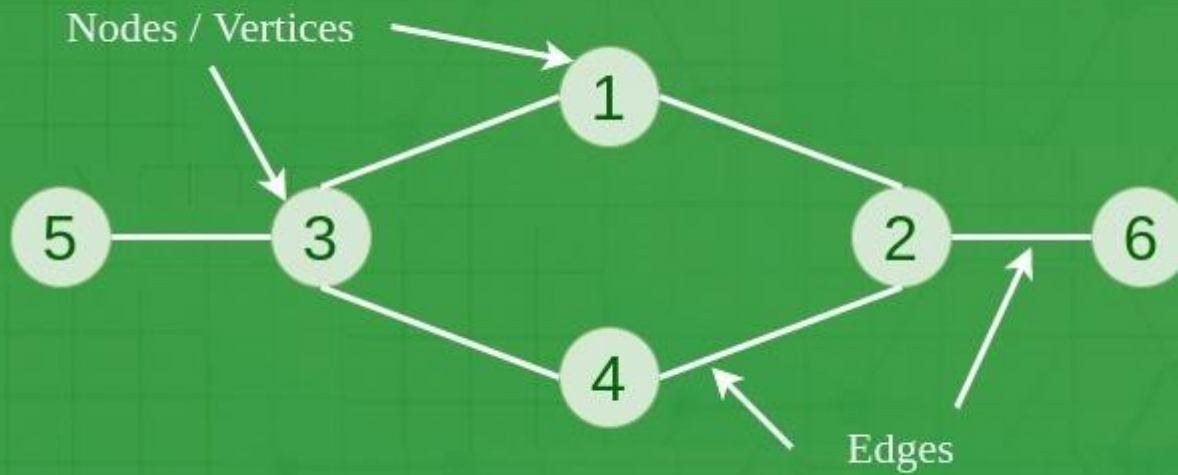
# Graph

- **Graph is collection of nodes (information) and connecting edges (Logical relation) between nodes.**
- **A tree can be viewed as restricted graph**
- **Graph have many types: 1) Simple graph 2) Mixed graph 3) Multi graph 4) Directed graph 5) Un-directed graph**

# Components of Graph



# Introduction to Graphs



# Components of a Graph

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labeled/unlabelled.

# Operations on Graph in Data Structure

- Following are the basic graph operations in data structure:
- Add/Remove Vertex – Add or remove a vertex in a graph.
- Add/Remove Edge – Add or remove an edge between two vertices.
- Check if the graph contains a given value.
- Find the path from one vertex to another vertex.

# Application of Graphs

- Graphs are used to represent flow of control in computers.
- Graphs are used in social networking sites where users act as nodes and connection between them acts as edges.
- In an operating system, graphs are used as resource allocation graphs.
- Graphs are used in Google maps to find the shortest route.
- Graphs are also used in airlines system for effective route optimization.
- In-state transition diagrams, the graph is used to represent their states and their transition.
- In transportation, graphs are used to find the shortest path.
- In circuits, graphs can be used to represent circuit points as nodes and wires as edges.
- Graphs are used in solving puzzles with only one solution, such as mazes.
- Graphs are used in computer networks for Peer to peer (P2P) applications.
- Graphs basically in the form of DAG(Directed acyclic graph) are used as alternative to blockchain for cryptocurrency. For example crypto like IOTA, Nano are mainly based on DAG.

## ALGORITHMS

**An Algorithm** is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.

From the data structure point of view, following are some important categories of algorithms –

- **Search** – Algorithm to search an item in a data structure.
- **Sort** – Algorithm to sort items in a certain order.
- **Insert** – Algorithm to insert item in a data structure.
- **Update** – Algorithm to update an existing item in a data structure.
- **Delete** – Algorithm to delete an existing item from a data structure.

### Characteristics of an Algorithm

Not all procedures can be called an algorithm. An algorithm should have the following characteristics –

- **Unambiguous** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- **Input** – An algorithm should have 0 or more well-defined inputs.

- **Output** – An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Feasibility** – Should be feasible with the available resources.
- **Independent** – An algorithm should have step-by-step directions, which should be independent of any programming code.

## How to Write an Algorithm?

There are no well-defined standards for writing algorithms. Rather, it is problem and resource dependent. Algorithms are never written to support a particular programming code.

As we know that all programming languages share basic code constructs like loops (do, for, while), flow-control (if-else), etc. These common constructs can be used to write an algorithm.

We write algorithms in a step-by-step manner, but it is not always the case. Algorithm writing is a process and is executed after the problem domain is well-defined. That is, we should know the problem domain, for which we are designing a solution.

## Example

Let's try to learn algorithm-writing by using an example.

**Problem** – Design an algorithm to add two numbers and display the result.

**Step 1 – START**

**Step 2 – declare three integers a, b & c**

**Step 3 – define values of a & b**

**Step 4 – add values of a & b**

**Step 5 – store output of step 4 to c**

**Step 6 – print c**

**Step 7 – STOP**

Algorithms tell the programmers how to code the program. Alternatively, the algorithm can be written as –

**Step 1 – START ADD**

**Step 2 – get values of a & b**

**Step 3 –  $c \leftarrow a + b$**

**Step 4 – display c**

**Step 5 – STOP**

In design and analysis of algorithms, usually the second method is used to describe an algorithm. It makes it easy for the analyst to analyze the algorithm ignoring all unwanted definitions. He can observe what operations are being used and how the process is flowing.

## **Algorithm Analysis**

Efficiency of an algorithm can be analyzed at two different stages, before implementation and after implementation. They are the following –

- **A Priori Analysis** – This is a theoretical analysis of an algorithm. Efficiency of an algorithm is measured by assuming that all other factors, for example, processor speed, are constant and have no effect on the implementation.
- **A Posterior Analysis** – This is an empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual statistics like running time and space required, are collected.

## Algorithm Complexity

Suppose **X** is an algorithm and **n** is the size of input data, the time and space used by the algorithm **X** are the two main factors, which decide the efficiency of **X**.

- **Time Factor** – Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
- **Space Factor** – Space is measured by counting the maximum memory space required by the algorithm.

The complexity of an algorithm **f(n)** gives the running time and/or the storage space required by the algorithm in terms of **n** as the size of input data.

## Space Complexity

Space complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle.

## ***Time Complexity***

Time complexity of an algorithm represents the amount of time required by the algorithm to run to completion. Time requirements can be defined as a numerical function  $T(n)$ , where  $T(n)$  can be measured as the number of steps, provided each step consumes constant time.

For example, addition of two  $n$ -bit integers takes  $n$  steps. Consequently, the total computational time is  $T(n) = c * n$ , where  $c$  is the time taken for the addition of two bits. Here, we observe that  $T(n)$  grows linearly as the input size increases.

Usually, the time required by an algorithm falls under three types –

- **Best Case** – Minimum time required for program execution.
- **Average Case** – Average time required for program execution.
- **Worst Case** – Maximum time required for program execution.

## **Asymptotic Notations**

Following are the commonly used asymptotic notations to calculate the running time complexity of an algorithm.

- ***O Notation***: The notation  $O(n)$  is the formal way to express the upper bound of an algorithm's running time. It measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete
- ***$\Omega$  Notation***: The notation  $\Omega(n)$  is the formal way to express the lower bound of an

algorithm's running time. It measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.

- **$\theta$  Notation:** The notation  $\theta(n)$  is the formal way to express both the lower bound and the upper bound of an algorithm's running time

### **Approaches of Algorithm design**

**The following are the approaches used after considering both the theoretical and practical importance of designing an algorithm:**

#### ***Brute force algorithm:***

The general logic structure is applied to design an algorithm. It is also known as an exhaustive search algorithm that searches all the possibilities to provide the required solution. Such algorithms are of two types:

**Optimizing:** Finding all the solutions of a problem and then take out the best solution or if the value of the best solution is known then it will terminate if the best solution is known.

**Sacrificing:** As soon as the best solution is found, then it will stop.

#### ***Greedy Algorithms***

An algorithm is designed to achieve optimum solution for a given problem. In greedy algorithm approach, decisions are made from the given solution domain. As being greedy,

the closest solution that seems to provide an optimum solution is chosen.

Greedy algorithms try to find a localized optimum solution, which may eventually lead to globally optimized solutions. However, generally greedy algorithms do not provide globally optimized solutions.

### ***Divide and Conquer***

In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently. When we keep on dividing the subproblems into even smaller sub-problems, we may eventually reach a stage where no more division is possible. Those "atomic" smallest possible sub-problem (fractions) are solved. The solution of all sub-problems is finally merged in order to obtain the solution of an original problem. Broadly, we can understand **divide-and-conquer** approach in a three-step process.

### ***Dynamic Programming***

Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems. But unlike, divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.

Dynamic programming is used where we have problems, which can be divided into similar sub-problems, so that their results can be re-used. Mostly, these algorithms are used for

optimization. Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems. The solutions of sub-problems are combined in order to achieve the best solution.

So we can say that –

- The problem should be able to be divided into smaller overlapping sub-problem.
- An optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- Dynamic algorithms use Memoization.

## **Comparison**

In contrast to greedy algorithms, where local optimization is addressed, dynamic algorithms are motivated for an overall optimization of the problem.

In contrast to divide and conquer algorithms, where solutions are combined to achieve an overall solution, dynamic algorithms use the output of a smaller sub-problem and then try to optimize a bigger sub-problem. Dynamic algorithms use Memoization to remember the output of already solved sub-problems

## *Types of Algorithms*

**The following are the types of algorithm:**

- **Search Algorithm**
- **Sort Algorithm**

### **Search Algorithm**

On each day, we search for something in our day to day life. Similarly, with the case of computer, huge data is stored in a computer that whenever the user asks for any data then the computer searches for that data in the memory and provides that data to the user. There are mainly two techniques available to search the data in an array:

- **Linear search**
- **Binary search**

### **Linear Search**

Linear search is a very simple algorithm that starts searching for an element or a value from the beginning of an array until the required element is not found. It compares the element to be searched with all the elements in an array, if the match is found, then it returns the index of the element else it returns -1. This algorithm can be implemented on the unsorted list.

## **Binary Search**

A Binary algorithm is the simplest algorithm that searches the element very quickly. It is used to search the element from the sorted list. The elements must be stored in sequential order or the sorted manner to implement the binary algorithm. Binary search cannot be implemented if the elements are stored in a random manner. It is used to find the middle element of the list.

# SORTING ALGORITHMS

# Sorting

- Sorting is a process in which records are arranged in ascending or descending order

|    |    |    |    |     |   |
|----|----|----|----|-----|---|
| 1  | 2  | 3  | 4  | 5   | 6 |
| 77 | 42 | 35 | 12 | 101 | 5 |



|   |    |    |    |    |     |
|---|----|----|----|----|-----|
| 1 | 2  | 3  | 4  | 5  | 6   |
| 5 | 12 | 35 | 42 | 77 | 101 |

# Types of sorting

- Selection sort
- Insertion sort
- Bubble sort
- Merge sort
- Quick sort
- Heap sort
- Shell sort

# Selection Sort

- **Selection sort** is a sorting algorithm which works as follows:
  - Find the minimum value in the list
  - Swap it with the value in the first position
  - Repeat the steps above for remainder of the list (starting at the second position)

# Example: Selection Sort

- 26 33 43 100 46 88 52 17 53 77
- 17 | 33 43 100 46 88 52 26 53 77
- 17 26 | 43 100 46 88 52 33 53 77
- 17 26 33 | 100 46 88 52 43 53 77
- 17 26 33 43 | 46 88 52 100 53 77
- 17 26 33 43 46 | 88 52 100 53 77
- 17 26 33 43 46 52 | 88 100 53 77
- 17 26 33 43 46 52 53 | 100 88 77
- 17 26 33 43 46 52 53 77 | 88 100
- 17 26 33 43 46 52 53 77 88 | 100

# Insertion Sort

- In insertion sort, each successive element in the array to be sorted is inserted into its proper place with respect to the other, already sorted elements.
- We divide our array in a sorted and an unsorted array
- Initially the sorted portion contains only one element: the first element in the array.
- We take the second element in the array, and put it into its correct place

# Insertion Sort

- That is, `array[0]` and `array[1]` are in order with respect to each other.
- Then the value in `array[2]` is put into its proper place, so `array [0].... array[2]` is sorted and so on.

|    |
|----|
| 36 |
| 24 |
| 10 |
| 6  |
| 12 |

|    |
|----|
| 36 |
| 24 |
| 10 |
| 6  |
| 12 |

|    |
|----|
| 24 |
| 36 |
| 10 |
| 6  |
| 12 |

|    |
|----|
| 10 |
| 24 |
| 36 |
| 6  |
| 12 |

|    |
|----|
| 6  |
| 10 |
| 24 |
| 36 |
| 12 |

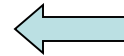
|    |
|----|
| 6  |
| 10 |
| 12 |
| 24 |
| 36 |

# Insertion Sort

- Our strategy is to search for insertion point from the beginning of the array and shift the element down to make room for new element
- We compare the item at `array[current]` to one before it, and swap if it is less.
- We then compare `array[current-1]` to one before it and swap if necessary.
-

# Example: Insertion Sort

- 99 | 55 4 66 28 31 36 52 38 72
- 55 99 | 4 66 28 31 36 52 38 72
- 4 55 99 | 66 28 31 36 52 38 72
- 4 55 66 99 | 28 31 36 52 38 72
- 4 28 55 66 99 | 31 36 52 38 72
- 4 28 31 55 66 99 | 36 52 38 72
- 4 28 31 36 55 66 99 | 52 38 72
- 4 28 31 36 52 55 66 99 | 38 72
- 4 28 31 36 38 52 55 66 99 | 72
- 4 28 31 36 38 52 55 66 72 99 |



# Bubble Sort

- Bubble sort is similar to selection sort in the sense that it repeatedly finds the largest/smallest value in the unprocessed portion of the array and puts it back.
- However, finding the largest value is not done by selection this time.
- We "bubble" up the largest value instead.

# Bubble Sort

- Compares adjacent items and exchanges them if they are out of order.
- Comprises of several passes.
- In one pass, the largest value has been “bubbled” to its proper position.
- In second pass, the last value does not need to be compared.

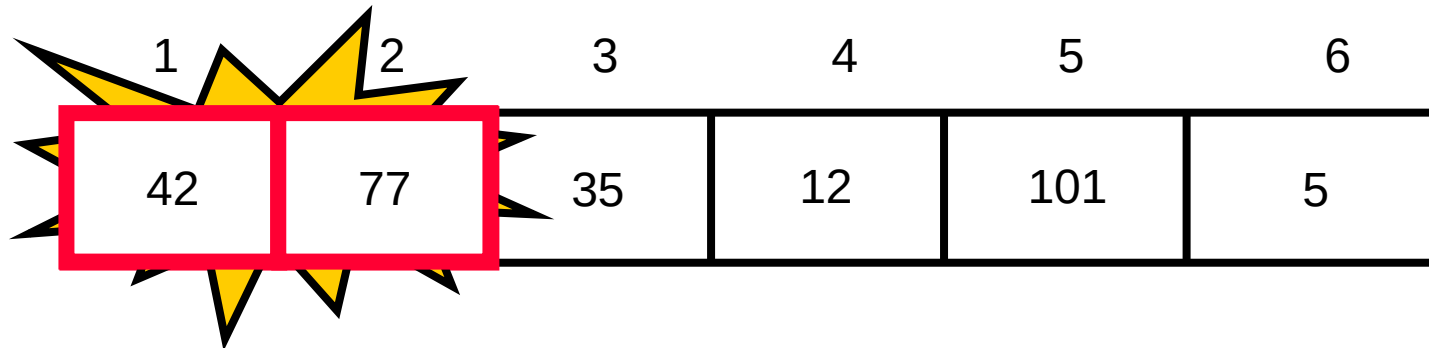
# Bubble Sort

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**

| 1  | 2  | 3  | 4  | 5   | 6 |
|----|----|----|----|-----|---|
| 77 | 42 | 35 | 12 | 101 | 5 |

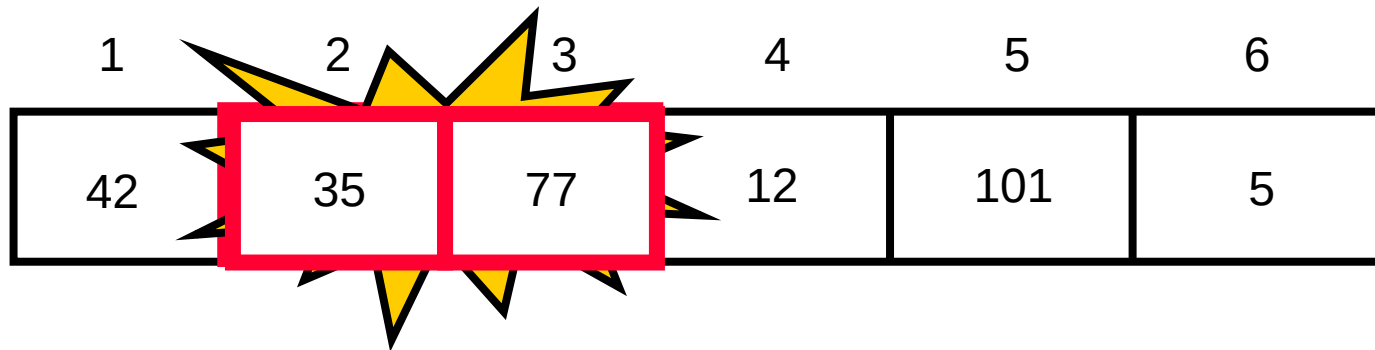
# Bubble Sort

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping



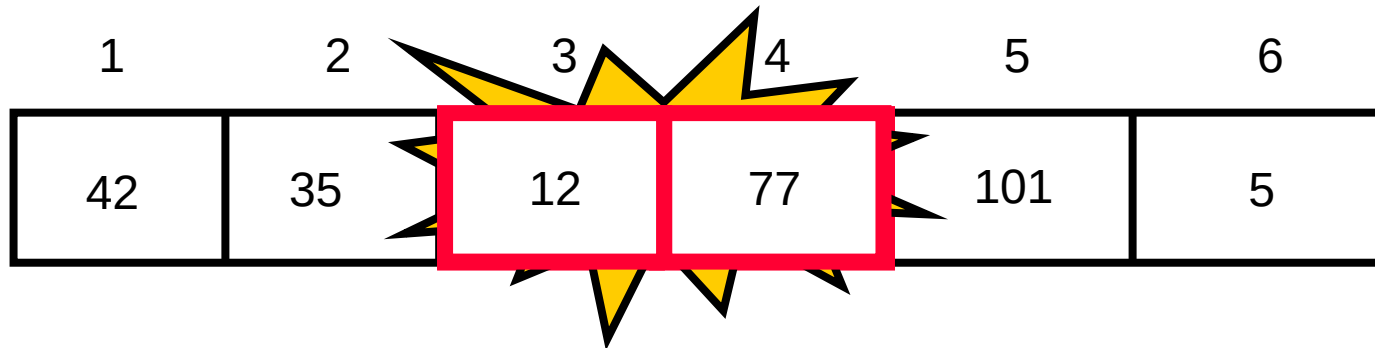
# Bubble Sort

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping



# Bubble Sort

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping



# Bubble Sort

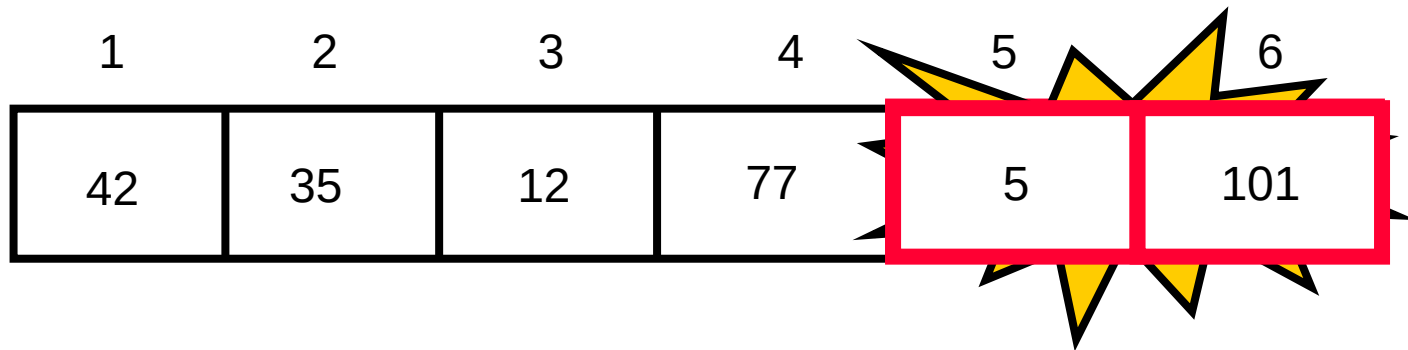
- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping

| 1  | 2  | 3  | 4  | 5   | 6 |
|----|----|----|----|-----|---|
| 42 | 35 | 12 | 77 | 101 | 5 |

No need to swap

# Bubble Sort

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping



# Bubble Sort

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping

|    |    |    |    |   |     |
|----|----|----|----|---|-----|
| 1  | 2  | 3  | 4  | 5 | 6   |
| 42 | 35 | 12 | 77 | 5 | 101 |

Largest value correctly placed

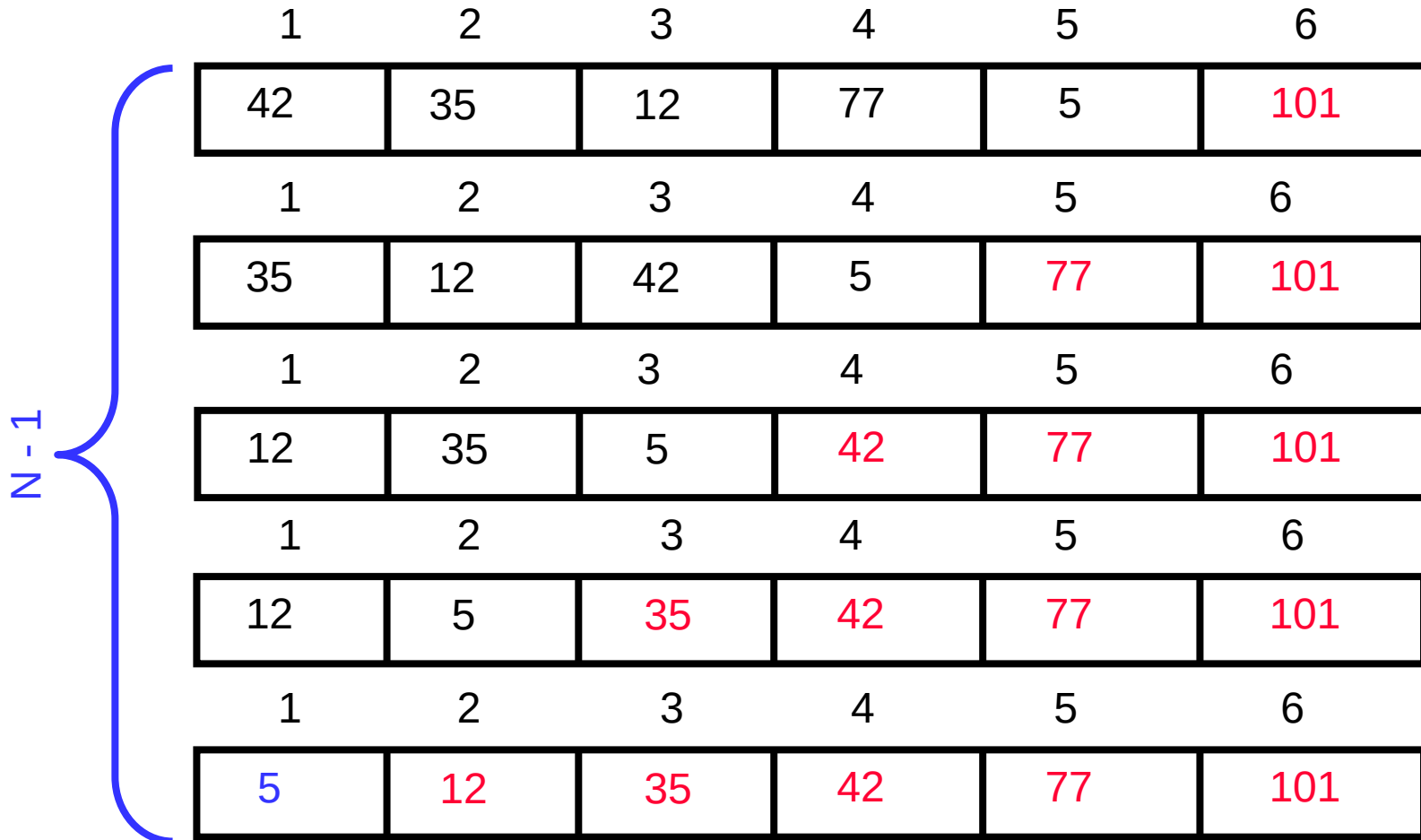
# Items of Interest

- Notice that only the largest value is correctly placed
- All other values are still out of order
- So we need to repeat this process

| 1  | 2  | 3  | 4  | 5 | 6   |
|----|----|----|----|---|-----|
| 42 | 35 | 12 | 77 | 5 | 101 |

Largest value correctly placed

# “Bubbling” All the Elements



# Reducing the Number of Comparisons

|    |    |    |    |     |     |
|----|----|----|----|-----|-----|
| 1  | 2  | 3  | 4  | 5   | 6   |
| 77 | 42 | 35 | 12 | 101 | 5   |
| 1  | 2  | 3  | 4  | 5   | 6   |
| 42 | 35 | 12 | 77 | 5   | 101 |
| 1  | 2  | 3  | 4  | 5   | 6   |
| 35 | 12 | 42 | 5  | 77  | 101 |
| 1  | 2  | 3  | 4  | 5   | 6   |
| 12 | 35 | 5  | 42 | 77  | 101 |
| 1  | 2  | 3  | 4  | 5   | 6   |
| 12 | 5  | 35 | 42 | 77  | 101 |

# Already Sorted Collections?

- What if the collection was already sorted?
- What if only a few elements were out of place and after a couple of “bubble ups,” the collection was sorted?
- We want to be able to detect this and “stop early”!

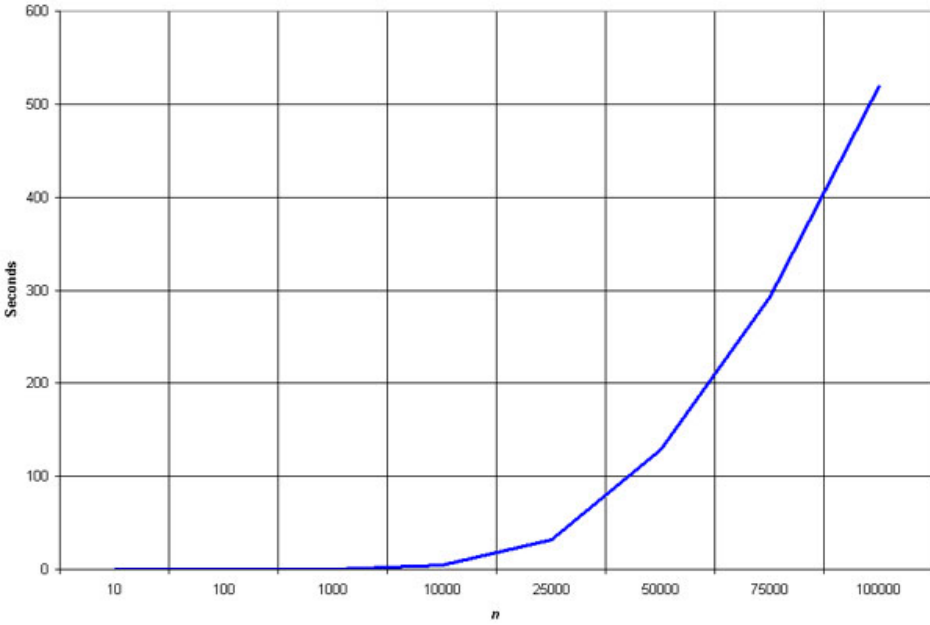
| 1 | 2  | 3  | 4  | 5  | 6   |
|---|----|----|----|----|-----|
| 5 | 12 | 35 | 42 | 77 | 101 |

# Using a Boolean “Flag”

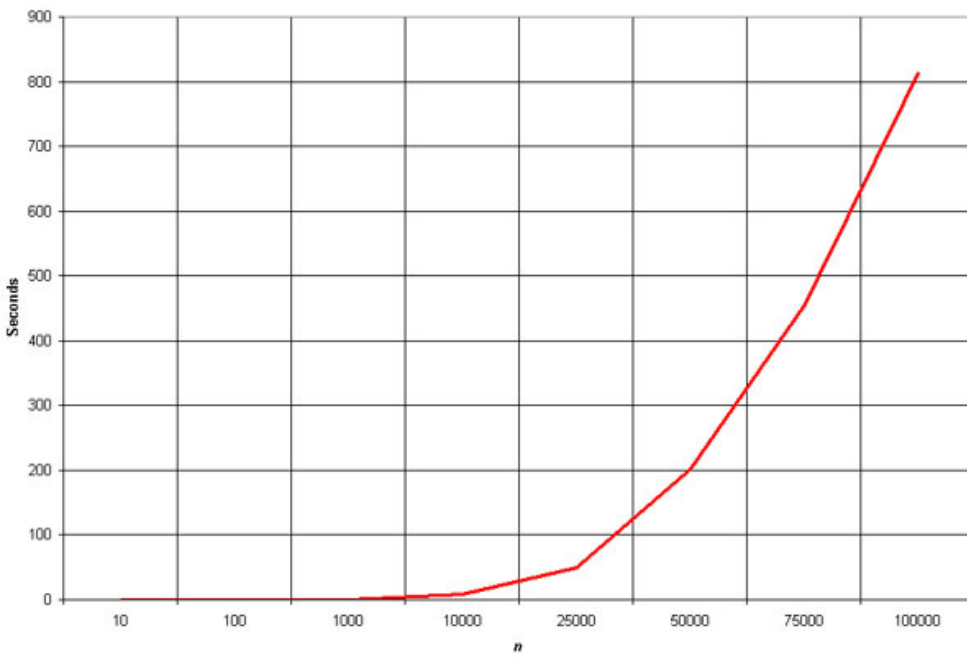
- We can use a boolean variable to determine if any swapping occurred during the “bubble up.”
- If no swapping occurred, then we know that the collection is already sorted!
- This boolean “flag” needs to be reset after each “bubble up.”

# Bubble Sort Algorithm

```
void bubbleSort (int S[ ], int length) {  
    bool isSorted = false;  
    while(!isSorted)  
    {  
        isSorted = true;  
        for(int i = 0; i<length; i++)  
        {  
            if(S[i] > S[i+1])  
            {  
                int temp = S[i];  
                S[i] = S[i+1];  
                S[i+1] = temp;  
                isSorted = false;  
            }  
        }  
        length--;  
    }  
}
```

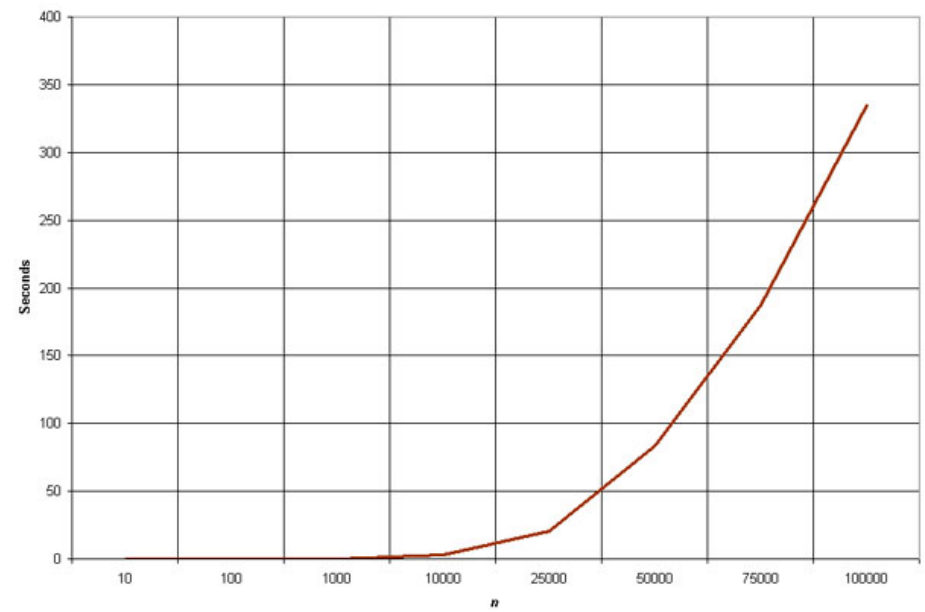


Selection Sort



Bubble Sort

Insertion Sort



# Summary

The insertion sort is a good middle-of-the-road choice for sorting lists of a few thousand items or less.

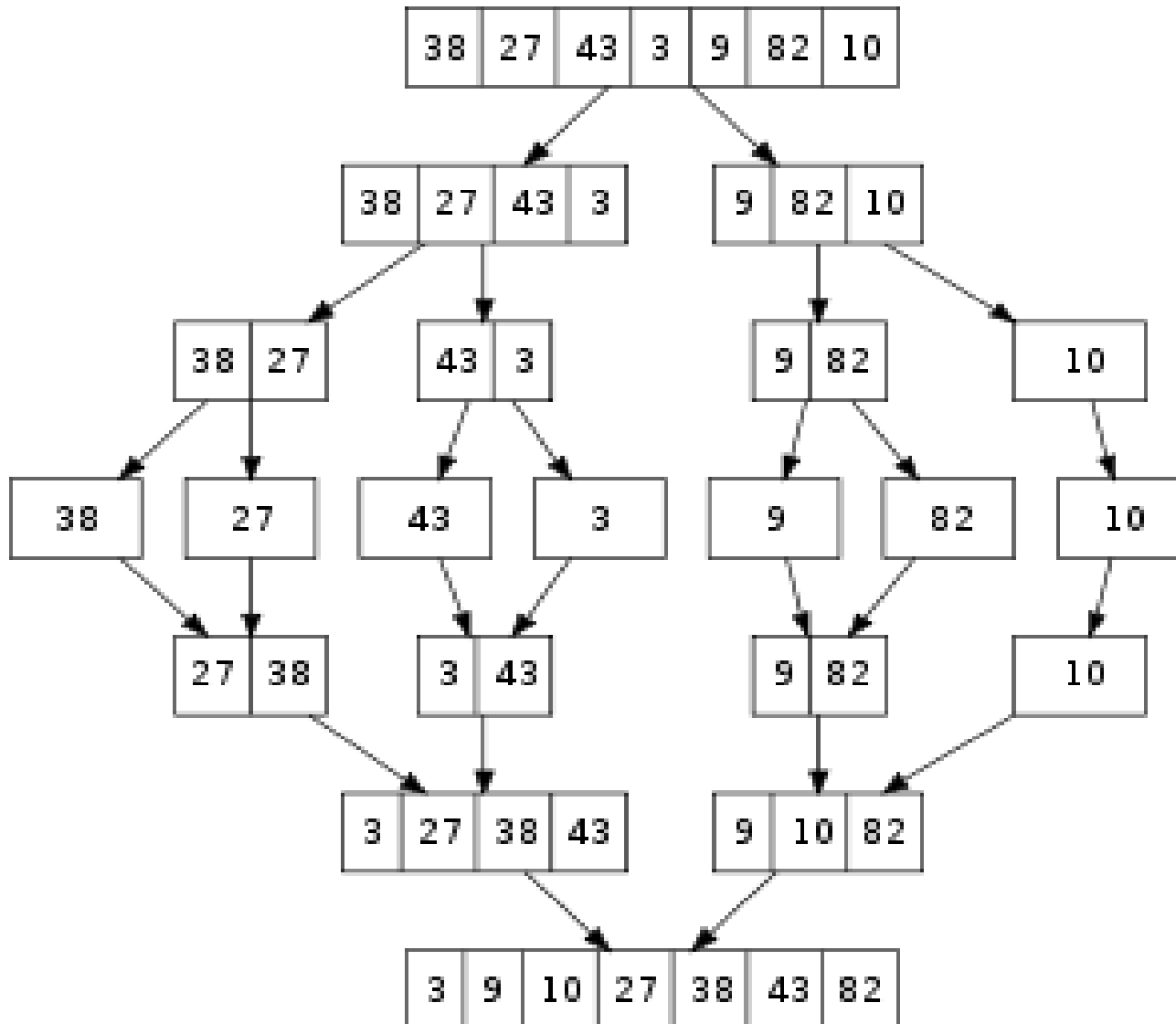
The insertion sort is over twice as fast as the bubble sort and almost 40% faster than the selection sort. The insertion sort shouldn't be used for sorting lists larger than a couple thousand items or repetitive sorting of lists larger than a couple hundred items.

# Mergesort

# Divide and Conquer

- **Divide and Conquer cuts the problem in half each time, but **uses the result of both halves**:**
  - cut the problem in half until the problem is trivial
  - solve for both halves
  - combine the solutions

# Merge Sort



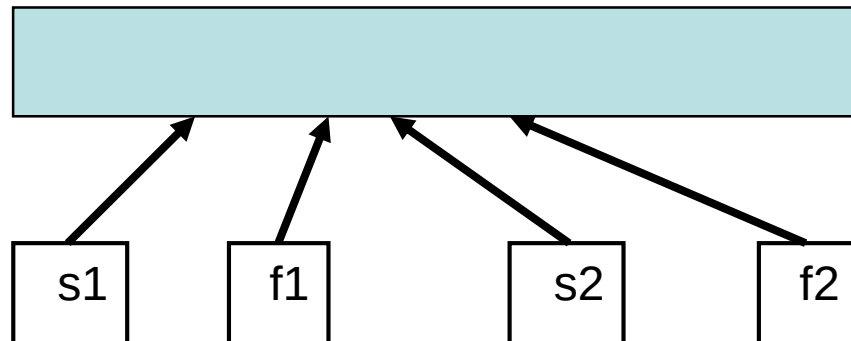
# Mergesort

- **A divide-and-conquer algorithm:**
- **Divide the unsorted array into 2 halves until the sub-arrays only contain one element**
- **Merge the sub-problem solutions together:**
  - **Compare the sub-array's first elements**
  - **Remove the smallest element and put it into the result array**
  - **Continue the process until all elements have been put into the result array**

|    |    |   |    |    |    |   |    |
|----|----|---|----|----|----|---|----|
| 37 | 23 | 6 | 89 | 15 | 12 | 2 | 19 |
|----|----|---|----|----|----|---|----|

# Merge Sort

- We don't really pass in two arrays!
- We pass in one array with indicator variables which tell us where one set of data starts and finishes and where the other set of data starts and finishes.



# Quicksort / partition-exchange sort

Quick sort is a divide and conquer algorithm.

Quick sort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quick sort can then recursively sort the sub-lists.

The steps are:

1. Pick an element, called a pivot, from the list.
2. Reorder the list so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the partition operation.
3. Recursively apply the above steps to the sub-list of elements with smaller values and separately the sub-list of elements with greater values.

**The base case of the recursion are lists of size zero or one, which never need to be sorted.**

# Quick Sort

5 4 9 7 1 8 3 2 6

4 1 3 2 5 9 7 8 6

1 3 2 4 5 7 8 6 9

1 3 2 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

  $\leq$    $<$  

## Quicksort

|    |   |   |    |    |    |    |    |   |
|----|---|---|----|----|----|----|----|---|
| 24 | 5 | 3 | 35 | 14 | 23 | 19 | 43 | 2 |
|----|---|---|----|----|----|----|----|---|

|   |   |   |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|
| 2 | 5 | 3 | 14 | 19 | 23 | 35 | 43 | 24 |
|---|---|---|----|----|----|----|----|----|

|   |   |   |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|
| 2 | 3 | 5 | 14 | 19 | 23 | 35 | 43 | 24 |
|---|---|---|----|----|----|----|----|----|

|   |   |   |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|
| 2 | 3 | 5 | 14 | 19 | 23 | 24 | 35 | 43 |
|---|---|---|----|----|----|----|----|----|

# Useful Links

<http://www.csanimated.com/animation.php?t=Quicksort>

<http://www.hakansozzer.com/category/c/>

<http://www.nczonline.net/blog/2012/11/27/computer-science-in-javascript-quicksort/>

<http://www.sorting-algorithms.com/shell-sort>